

# Practico-G2-Segundo-Parcial.pdf



andreagb03



Inteligencia Artificial



3º Grado en Ingeniería Informática - Tecnologías Informáticas



Escuela Técnica Superior de Ingeniería Informática  
Universidad de Sevilla



[Accede al documento original](#)

**CURSO NUEVO  
EQUIPO NUEVO**

**CeX**  
webuy.com

6+7=



**CURSO  
NUEVO  
EQUIPO  
NUEVO**

27/12/24, 12:20

Práctico G2 Segundo Parcial

## Evaluación Práctica - 2do Parcial (Grupo 2)

Inteligencia Artificial - G.I.I. Tecnologías Informáticas -  
Curso 2022/23

Departamento de Ciencias de la Computación e Inteligencia Artificial

Universidad de Sevilla

Fecha: 22 de diciembre de 2022

### Instrucciones de la prueba

#### 1. Ponga su nombre, apellidos y uvus.

```
In [ ]: # Nombre:  
        # Apellidos:  
        # UVUS:
```

#### 2. Renombre este archivo cambiando UVUS por su UVUS

Para ello, pulse sobre el nombre `IATI2223_EP2_G2__UVUS` (arriba a la izquierda junto al logo de jupyter) y cambie UVUS por el suyo. Por ejemplo, si el uvus fuese `profiati` el archivo quedaría como `IATI2223_EP2_G2__profiati`

#### 3. Complete el examen.

En cada uno de los apartados se da el enunciado y un conjunto de ejemplos de prueba junto con la salida esperada para testear si dicha salida coincide con la salida de su solución.

Lea los enunciados con atención antes de intentar completar los ejercicios.

#### 4. Entrega del examen

Cree una carpeta en el **ESCRITORIO** con el nombre `IATI2223_EP2_G2__UVUS` de este fichero (después de haber sido renombrado). Comprima la carpeta en `.tar.gz` y asegúrese de que no existan otros archivos en el Escritorio con dicha extensión.

**IMPORTANTE: ASEGURARSE QUE EL CONTENIDO DE LA CARPETA COMPRIMIDA ES EL QUE SE QUIERE ENVIAR.**



WUOLAH

Una vez comprimido (y asegurada la corrección) apague el ordenador y vuélvalo a encender hasta llegar otra vez al menú blanco. Pulsar en "Enviar examen" y solicitar al profesor que compruebe la corrección del envío.

ESTO ÚLTIMO DEMUESTRA QUE EL FICHERO DEL ESCRITORIO HA LLEGADO CORRECTAMENTE, PERO NO HACE COMPROBACIONES SOBRE EL CONTENIDO DEL MISMO. **ES RESPONSABILIDAD DEL ALUMNO ASEGURARSE DE QUE LO QUE HA ENVIADO ES CORRECTO**

## Ejercicios del examen

```
In [7]: import random
from copy import deepcopy
import iris
```

### EJERCICIO 1. (2.75 pts)

Considérese una red neuronal multicapa, dada por una clase que goza de las siguientes propiedades:

| Propiedad | Tipo                                  | Descripción                                                                                                                                                                                                                      |
|-----------|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| entrada   | list:int                              | Representa los índices de las neuronas de la capa de entrada de la red.                                                                                                                                                          |
| ocultas   | list:<br>(list:int,function,function) | Representa las distintas capas ocultas de la red, mediante ternas (ns_c, g_c, gp_c) correspondientes a una lista con los índices de las neuronas de la capa, la función de activación de la misma y su derivada.                 |
| salida    | (list:int,function,function)          | Representa la capa de salida mediante una terna (igual a la de las capas ocultas), (ns_c, g_c, gp_c) correspondiente a una lista con los índices de las neuronas de la capa, la función de activación de la misma y su derivada. |
| pesos     | dict:(int,int) -> float               | Representa los pesos de la red, asociándolos a cada una de las aristas de la misma, dadas por la pareja de los índices de las neuronas correspondientes.                                                                         |
| umbrales  | dict:int -> float                     | Representa los pesos umbrales de la red, asociándolos a cada una de las neuronas (que no son de entrada), dadas por sus índices.                                                                                                 |

Así la siguiente red estaría representada por:



No description has been provided for this image

ENTRADA\_EJ=[1,2,3,4] OCULTAS\_EJ= [( [5,6], relu, relup ), ( [7,8], relu, relup )] SALIDA\_EJ=[9,10,11], sigmoide, sigmoidep) PESOS\_EJ = { (1,5): 0.15341236, (2,5): -1.19818626, (3,5): 1.11221752, (4,5): 2.97788312, (1,6): 0.95729355,

(2,6): 1.4438379, (3,6): 0.63808725, (4,6): -0.35773559, (5,7): 0.6504594, (6,7): -0.56082912, (5,8): 3.18479707, (6,8): -1.32189382, (7,9): 0.86299307, (8,9): -2.91966467, (7,10): 0.97772277, (8,10): -0.07014417, (7,11): 0.64382918, (8,11): 0.62495951 } UMBRALES\_EJ = { 5: 0.36010888, 6: 0.8381687, 7: 0.69783656, 8: 0.28975424, 9: 2.26844483, 10: 0.01051107, 11: -4.82982895 }

## APARTADO A (0.5 ptos)

Se pide implementar una función `pesosIniciales(entrada, ocultas, salida)` que devuelva dos diccionarios estableciendo cada uno de los pesos de la red con un número aleatorio entre -0.5 y 0.5 (se recomienda el uso de `random.uniform`); e igualmente para los umbrales correspondientes.

```
In [11]: def pesosIniciales(entrada, ocultas, salida):
# Comenzamos con Los diccionarios vacíos para los pesos y los umbrales
pesos, umbrales = dict(), dict()
### COMPLETAR ###
# pesos de entrada a primera capa oculta
for entrada_neurona in entrada:
    for oculta_neurona in ocultas[0][0]:
        pesos[(entrada_neurona, oculta_neurona)] = random.uniform(-0.5, 0.5)
# pesos entre capas ocultas consecutivas
for i in range(len(ocultas)-1):
    for neurona_actual in ocultas[i][0]:
        for neurona_siguiente in ocultas[i+1][0]:
            pesos[(neurona_actual, neurona_siguiente)] = random.uniform(-0.5,
# pesos última oculta a salida
for oculta_neurona in ocultas[-1][0]:
    for salida_neurona in salida:
        pesos[(oculta_neurona, salida_neurona)] = random.uniform(-0.5, 0.5)
# umbrales para todas las neuronas q no son de entrada
for capa in ocultas:
    for neurona in capa[0]:
        umbrales[neurona] = random.uniform(-0.5, 0.5)
for neurona in salida[0]:
    umbrales[neurona] = random.uniform(-0.5, 0.5)
return pesos, umbrales
```

```
In [13]: # EJECUTAR SÓLO COMO COMPROBACIÓN
from examen_sources import redIris

random.seed(1)
pesosInit, umbralesInit = pesosIniciales(redIris.entrada, redIris.ocultas, redIris.salida)

print(f"pesos={pesosInit}\n\numbrales={umbralesInit}")

# SALIDA ESPERADA:
# pesos={
#   (1, 5): -0.3656357558875988, (1, 6): 0.3474337369372327,
#   (2, 5): 0.26377461897661403, (2, 6): -0.2449309742605783,
#   (3, 5): -0.004564912908059049, (3, 6): -0.050508935211261874,
#   (4, 5): 0.15159297272276295, (4, 6): 0.2887233511355132,
#   (5, 7): 0.3357651039198697, (5, 8): -0.06723293209494663,
#   (6, 7): 0.262280082457942, (6, 8): -0.4978939466488893,
#   (7, 9): -0.27123777872954735, (7, 10): 0.4452706955539223, (7, 11): 0.4014
#   (8, 9): -0.46941001696644646, (8, 10): -0.4745541390065392, (8, 11): 0.041
# }
```



$$6+7=$$



**CURSO  
NUEVO  
EQUIPO  
NUEVO**

27/12/24, 12:20

Práctico G2 Segundo Parcial

```
# umbrales={
#     5: -0.4061404132257651, 6: -0.4716525234779937,
#     7: -0.05461280594519857, 8: 0.22154003234078257,
#     9: 0.43914916277851057, 10: -0.11879576231178757, 11: -0.2834006028693866}
```

```
-----
ImportError                                Traceback (most recent call last)
Cell In[13], line 2
      1 # EJECUTAR SÓLO COMO COMPROBACIÓN
----> 2 from examen_sources import redIris
      4 random.seed(1)
      5 pesosInit, umbralesInit = pesosIniciales(redIris.entrada, redIris.oculta
s, redIris.salida)

ImportError: bad magic number in 'examen_sources': b'a\r\r\n'
```

## APARTADO B (0.75 pto)

Se pide implementar una función `forward(e_x, ENTRADA, OCULTAS, SALIDA, PESOS, UMBRALES)` que, dado un ejemplo `e_x`, realice el paso de propagación hacia adelante, y devuelva `valores_in` y `valores_a`, correspondientes a dos diccionarios que almacenan, para cada neurona (dada por el índice), su valor de entrada y valor de activación, respectivamente.

Para ello:

```
In [15]: from copy import deepcopy

def forward(x, ENTRADA, OCULTAS, SALIDA, PESOS, UMBRALES):
    # 0. Guardamos en variables algunos elementos que pueden resultar relevantes
    entrada = deepcopy(ENTRADA)
    ocultas = deepcopy(OCULTAS)
    salida = deepcopy(SALIDA)
    pesos = deepcopy(PESOS)
    umbrales = deepcopy(UMBRALES)

    # 1.1 Inicializamos el diccionario que almacenará la salida de las neuronas.

    valores_a = dict(zip(entrada, x))

    # 1.2 Inicializamos el diccionario que almacenará los valores de entrada de
    #       se necesita almacenar para la capa de entrada

    valores_in = dict()

    ### IMPLEMENTAR EL SIGUIENTE ALGORITMO ###

    # 2. Propagar las activaciones a través de las capas ocultas.

    # 2.1 (Opcional) Almacenamos en una variable las neuronas de la última capa
    #               las neuronas de la capa de entrada:
    ns_lprev = entrada[:]

    # 2.2 Para cada capa oculta hacer:
    for capa in ocultas:
```



webuy.com



WUOLAH

```

# 2.2.1 Para cada neurona de la capa, hacer
neuronas_capa, g_c, _ = capa #obtenemos las neuronas y la función de act
for neurona in neuronas_capa:
    # (-a-)  $in[i] \leftarrow umbral[i] + \sum (w[j,i] * a[j])$ ; con j cada una de la
    valores_in[neurona]=umbrales[neurona]+sum(pesos.get((neurona_prev, n
    # (-b-)  $a[i] \leftarrow g_c(in[i])$ 
    valores_a[neurona]=g_c(valores_in[neurona])
# 2.2.2 (Si se utiliza ns_lprev) Antes de pasar a la siguiente capa, act
#
ns_lprev = neuronas_capa[:]

# 3. Propagar las activaciones a través de la capa de salida
# 3.1 Para cada neurona de la capa de salida, hacer:
neuronas_salida, g_c, _=capa
for neurona in neuronas_salida:
    #  $in[i] \leftarrow umbral[i] + \sum (w[j,i] * a[j])$ ; con j cada una de las neuronas
    valores_in[neurona]=umbrales[neurona]+sum(pesos.get((neurona_prev, neuro
    #  $a[i] \leftarrow g_c(in[i])$ 
    valores_a=g_c(valores_in[neurona])
# 4. Devolvemos los diccionarios con los valores de entrada y de salida

return valores_in, valores_a

```

```

In [17]: # EJECUTAR ÚNICAMENTE COMO COMPROBACIÓN
from iris import irisX
from examen_sources import redIris, pesosInicialesEx

pesosInit, umbralesInit = pesosInicialesEx(redIris.entrada, redIris.ocultas, redIris.salida)

in_dict, a_dict = forward(irisX[0], redIris.entrada, redIris.ocultas, redIris.salida, pesosInit, umbralesInit)
print(f"valores_in={in_dict};\n\nvalores_a={a_dict};")

# Resultado esperado
# valores_in={5: -1.4477167855357886, 6: 1.2929968959761329, 7: 0.28451452654928
#
#           9: 0.3619780745809938, 10: 0.007890218820007289, 11: -0.1691886598
#
# valores_a={1: 5.8, 2: 2.8, 3: 5.1, 4: 2.4, 5: 0, 6: 1.2929968959761329, 7: 0.2
#
#           9: 0.5895191856572395, 10: 0.5019725444715252, 11: 0.45780344273298

```

```

-----
ImportError                                Traceback (most recent call last)
Cell In[17], line 3
      1 # EJECUTAR ÚNICAMENTE COMO COMPROBACIÓN
      2 from iris import irisX
----> 3 from examen_sources import redIris, pesosInicialesEx
      4 pesosInit, umbralesInit = pesosInicialesEx(redIris.entrada, redIris.ocultas, redIris.salida)
      5 in_dict, a_dict = forward(irisX[0], redIris.entrada, redIris.ocultas, redIris.salida, pesosInit, umbralesInit)

ImportError: bad magic number in 'examen_sources': b'a\r\r\n'

```

## APARTADO B (0.75 pto)

Se pide implementar una función `backward(e_y, valores_in, valores_a, tasa_ap, ENTRADA, OCULTAS, SALIDA, PESOS, UMBRALES)` que dados los valores

# INTERCAMBIA y AHORRA en LA VUELTA A LA UNI



[webuy.com](https://www.webuy.com)



de clasificación reales correspondientes a un ejemplo ( `e_y` ), los valores de entrada ( `valores_in` ) y activación ( `valores_a` ) de las neuronas en el momento actual, los elementos que conforman la red, y la tasa de aprendizaje ( `eta` ); ejecute el paso de propagación hacia atrás y devuelva un diccionario con los pesos actualizados y otro con los umbrales actualizados.

```
In [45]: from copy import deepcopy
def backward(e_y, valores_in, valores_a, eta, ENTRADA, OCULTAS, SALIDA, PESOS, UMBRALES):
    # 0.1 Guardamos en variables algunos elementos que pueden resultar relevante
    entrada = deepcopy(ENTRADA)
    ocultas = deepcopy(OCULTAS)
    salida = deepcopy(SALIDA)
    pesos = deepcopy(PESOS)
    umbrales = deepcopy(UMBRALES)
    # Almacenamos en las variables y, o dos diccionarios que corresponden a los valores
    # de salida esperados y valores reales, respectivamente, en cada una de las
    # de la capa de salida.
    y = dict(zip(salida[0], e_y))
    o = {i:valores_a[i] for i in salida[0]}
    # 0.2 Inicializamos los diccionarios que almacenarán los resultados y también
    # errores en cada una de las neuronas
    nuevos_pesos, nuevos_umbrales, delta = dict(), dict(), dict()
    ### IMPLEMENTAR EL SIGUIENTE ALGORITMO ###
    # 0. (Opcional) Definir las variables ns_l y gp_l para almacenar las neuronas
    # de la capa actual, en este momento, la de salida.
    ns_l, _, gp_l = salida
    # 1. Para cada unidad i en la capa de salida hacer:
    for neurona in ns_l:
        # 1.1. Calcular el error en cada neurona, como:  $\Delta[i] \leftarrow g'_l(in[i]) * (y[i] - o[i])$ 
        delta[neurona] = gp_l(valores_in[neurona] * (y[neurona] - o[neurona]))
        # 1.2. Actualizar el umbral de la unidad como:  $umbral[i] \leftarrow umbral[i] + \eta * \Delta[i]$ 
        nuevos_umbrales[neurona] = umbrales[neurona] + eta * delta[neurona]
    # 2. (Opcional) Utilizar una variable que almacene las neuronas de la última
    # adelante, el valor de dicha variable corresponderá a las neuronas de
    # salida.
    ns_lsig = salida[0]
    # 2. Para cada una de las capas de ocultas, del final al inicio:
    for k in range(len(ocultas)-1, -1, -1):
        ns_l, _, gp_l = ocultas[k]
        # 2.1 Para cada neurona j de la capa, hacer:
        for j in ns_l:
            # 2.1.0. Inicializar el error  $\Delta[j]$  a 0 (iremos añadiendo el error en
            delta[j] = 0
            # 2.1.1. Para cada neurona i de la capa siguiente:
            for i in ns_lsig:
                # (-a-) Añadir a  $\Delta[j]$  la contribución al error de i debido a j,
                # (-b-) Actualizar el peso  $w[j,i]$  como:  $w[j,i] \leftarrow w[j,i] + \eta * a[i] * \Delta[j]$ 
                nuevos_pesos[j,i] = pesos[j,i] + eta * valores_a[i] * delta[j]
            # 2.1.2 Actualizar el umbral[j] como:  $umbral[j] \leftarrow umbral[j] + \eta * \Delta[j]$ 
            nuevos_umbrales[j] = umbrales[j] + eta * delta[j]
        # 2.2. (Si se usa ns_lsig) actualizar su valor con las neuronas de la capa
        ns_lsig = ns_l
    # 3. Para cada neurona j de la capa de entrada:
    for j in entrada:
        # 3.1. Para cada neurona i de la capa siguiente (o sea, de la primera capa
        for i in ocultas[0][0]:
            # 3.1.1. Actualizar el peso  $w[j,i]$  como:  $w[j,i] \leftarrow w[j,i] + \eta * a[i] * \Delta[j]$ 
            nuevos_pesos[j,i] = pesos[j,i] + eta * valores_a[i] * delta[j]
```



$$6+7=$$



**CURSO  
NUEVO  
EQUIPO  
NUEVO**

27/12/24, 12:20

Práctico G2 Segundo Parcial

```
#4. Devolver Los nuevos pesos y Los nuevos umbrales.  
return nuevos_pesos, nuevos_umbrales
```

```
In [47]: # EJECUTAR SÓLO COMO COMPROBACIÓN  
from iris import irisX, irisY_OH  
from examen_sources import redIris, pesosInicialesEx, forwardEx  
  
pesosInit, umbralesInit = pesosInicialesEx(redIris.entrada, redIris.ocultas, redIris.salida,  
in_dict, a_dict = forwardEx(irisX[0], redIris.entrada, redIris.ocultas, redIris.salida)  
  
pesosAct_dict, umbralesAct_dict = backward(irisY_OH[0], in_dict, a_dict, 0.001,  
redIris.entrada, redIris.ocultas, redIris.salida)  
  
print(f"nuevos_pesos={pesosAct_dict};\n\nnuevos_umbrales={umbralesAct_dict};")  
  
# RESPUESTA ESPERADA:  
# nuevos_pesos={  
# (7, 9): 0.8629524824164502, (7, 10): 0.9776870659354892, (7, 11): 0.643867  
# (8, 9): -2.91966467, (8, 10): -0.07014417, (8, 11): 0.62495951,  
# (5, 7): 0.6504594, (5, 8): 3.18479707,  
# (6, 7): -0.561034910058628, (6, 8): -1.3218872332371698,  
# (1, 5): 0.15340729651270463, (1, 6): 0.9577722016727778,  
# (2, 5): -1.1981887044421426, (2, 6): 1.4440689732213408,  
# (3, 5): 1.1122130676232402, (3, 6): 0.6385081333674425,  
# (4, 5): 2.9778810247638776, (4, 6): -0.35753752723885057};  
  
# nuevos_umbrales={9: 2.2683021744244054, 10: 0.010385578817023, 11: -4.82969436  
# 7: 0.6976774025733515, 8: 0.2897593341830183,  
# 5: 0.3601080069849491, 6: 0.8382512261504789};
```

Traceback (most recent call last):

```
File ~\anaconda3\Lib\site-packages\IPython\core\interactiveshell.py:3577 in run_code  
exec(code_obj, self.user_global_ns, self.user_ns)  
  
Cell In[47], line 3  
from examen_sources import redIris, pesosInicialesEx, forwardEx  
  
File <string>  
SyntaxError: source code string cannot contain null bytes
```

## APARTADO D (0.75 ptos)

UTILIZANDO LAS FUNCIONES ANTERIORES, se pide implementar una clase `FNN` con las propiedades especificadas en el ejercicio, completando los siguientes métodos propios de la clase.

Utilizar las funciones `pesosInicialesEx`, `forwardEx`, `backwardEx`, que corresponden a las soluciones aportadas por los profesores para los apartados correspondientes.

```
In [ ]: from examen_sources import barajaConjuntoDatos, pesosInicialesEx, forwardEx, backwardEx  
  
class FNN():  
    def __init__(self, ENTRADA, OCULTAS, SALIDA, PESOS=None, UMBRALES=None):  
        self.entrada = ENTRADA
```

```

self.ocultas = OCULTAS
self.salida = SALIDA
self.pesos = PESOS
self.umbrales = UMBRALES

def train(self, trainX, trainY, eta, epochs):

    ### IMPLEMENTAR ESTE MÉTODO ###

    # 1. Si Los pesos/umbrales son None, entonces se inicializan aleatoriamente
    if self.pesos == None or self.umbrales == None:
        self.pesos = random.random()
    # 2. Repetir durante el número de epochs:

    # 2.1. Reordenar aleatoriamente el conjunto de ejemplos:
    datosX, datosY = barajaConjuntoDatos(trainX, trainY)

    # 2.2 Para cada ejemplo (x,y), hacer: (Ayuda: tomar x,y de trainX y

    # 2.2.1. Propagar hacia adelante (forward), obteniendo los valores
    #         una de las neuronas
    valores_in, valores_a = ???

    # 2.2.2. Propagar hacia atrás (backward), calculando los errores
    nuevos_pesos, nuevos_umbrales = ???

    # 2.2.3. Actualizar los pesos y umbrales de la red con los obtenidos
    self.pesos, self.umbrales = ???

def predict(self, e_x):
    # 1. Propagar hacia adelante (forward), obteniendo los valores de entrada
    valores_in, valores_a = ???

    # 2. Calcular los valores de salida de la red
    o = ???

    # 3. Devolver dichos valores
    return o

```

## EJERCICIO 2 (0.75 pts)

En este apartado se pide realizar un flujo de prueba completo sobre el conjunto de datos Iris, que posee:

- `irisX` : Una lista con los ejemplos (atributos).
- `irisY_OH` : Una lista con los valores de clasificación de los ejemplos, en codificación ONE-HOT.

```
In [ ]: from iris import irisX, irisY_OH
```

1. (0.1 pts) Partir el conjunto de datos en dos subconjuntos de entrenamiento (propTrain:0.8) y prueba.

NOTA: Utilizar la función `separaDatosTrainTestEx(datosX, datosY, propTrain=0.8)`

```
In [ ]: from examen_sources import separaDatosTrainTestEx

trainX, trainY, testX, testY = ???
```

2. (0.15 ptos) Instanciar la siguiente red neuronal `FNN`, sin pesos ni umbrales inicializados.

NOTA: Definirla utilizando la clase `FNNEx` (que corresponde a la solución aportada por los profesores al ejercicio 1D). Utilizar las funciones de activación `sigmoide`, `relu`, y sus derivadas `sigmoidep`, `relup`;

No description has been provided for this image

```
In [ ]: from examen_sources import sigmoide, sigmoidep, relu, relup, FNNEx

redE2 = ???
```

3. (0.1 ptos) Entrenar la red con el conjunto de entrenamiento, utilizando 2500 epochs, eta=0.001.

```
In [ ]: ### COMPLETAR ###
```

4. (0.4 ptos) Calcular la tasa de acierto sobre el conjunto de test del modelo neuronal.

NOTA: Puede resultar útil la función `argmax`, que dada una lista devuelve el índice cuya posición posee un mayor valor.

```
In [ ]: from numpy import argmax

# Inicializamos un contador que vaya marcando el número de aciertos
P = 0

# Guardamos en una variable el número de ejemplos del conjunto de test
n = len(testX)

### COMPLETAR ###

# Para cada ejemplo:

    # Obtenemos el valor de clasificación real

    # Obtenemos el valor de clasificación dado por la red

    # Contamos el acierto si y sólo si el valor predicho es igual al real

print(f"\nTest Resultado:{P}/{n}={P/n}")
```

$$6+7=$$



**CURSO  
NUEVO  
EQUIPO  
NUEVO**

27/12/24, 12:20

Práctico G2 Segundo Parcial

```
In [ ]: # Resultado esperado
from examen_sources import testIrisEx

testIrisEx(testX, testY, redE2)
```



webuy.com



WUOLAH